

Aprendizaje por Refuerzo

Alberto López Corbalán
`alberto.lopezc@um.es`

17 de junio de 2026

Resumen

La toma de decisiones en entornos inciertos constituye uno de los problemas fundamentales dentro del ámbito de la inteligencia artificial. Con el objetivo de comprender los principios fundamentales y las principales técnicas empleadas en aprendizaje por refuerzo, este trabajo se estructura en dos bloques.

En primer lugar, se estudia el problema del Bandido Multi-Brazo, un modelo simplificado que permite analizar de forma aislada el dilema exploración-explotación. A través de este problema se introducen y evalúan diferentes estrategias de selección de acciones, observando su comportamiento bajo distintos modelos de recompensa.

En segundo lugar, se estudian entornos de aprendizaje por refuerzo de mayor complejidad utilizando la biblioteca Gymnasium. Dentro de este bloque se distinguen dos enfoques principales. En primer lugar, se analizan métodos tabulares aplicados a entornos discretos, donde las funciones de valor pueden representarse explícitamente mediante tablas. En segundo lugar, se abordan técnicas de aproximación de funciones destinadas a resolver problemas con espacios de estados continuos, en los que las representaciones tabulares dejan de ser viables.

El siguiente repositorio que contiene el código fuente y documentación asociados al trabajo: <https://github.com/AlbertoLopezCorbalan/eml-albertolopezc/blob/main/main.ipynb>

Índice general

1. Problema del Bandido de k-brazos	2
1.1. Introducción	2
1.2. Desarrollo	2
1.2.1. Métricas de evaluación	2
1.3. Algoritmos	3
1.3.1. ϵ -greedy	3
1.3.2. Upper Confidence Bound (UCB1)	3
1.3.3. Softmax	3
1.4. Evaluación/Experimentos	3
1.5. Conclusiones	5
2. Aprendizaje en Entornos Complejos	6
2.1. Introducción	6
2.2. Desarrollo	6
2.2.1. Entorno Taxi-v4	6
2.2.2. Entorno LunarLander-v3	6
2.2.3. Métricas de evaluación	7
2.3. Algoritmos	7
2.3.1. Monte Carlo	7
2.3.2. SARSA/SARSA Semi-Gradient	7
2.3.3. Q-Learning/Deep Q-Learning	8
2.4. Evaluación/Experimentos	8
2.4.1. Tabular: Taxi-V4	8
2.4.2. Continuo: LunarLander-V3	9
2.5. Conclusiones	10

Capítulo 1

Problema del Bandido de k -brazos

1.1. Introducción

El problema del Bandido de k -brazos (*Multi-Armed Bandit*) constituye uno de los problemas clásicos dentro del aprendizaje por refuerzo y la teoría de la decisión secuencial. El problema recibe su nombre de la analogía con una máquina tragaperras que dispone de varios brazos, cada uno asociado a una distribución de recompensas desconocida. El objetivo del agente consiste en seleccionar sucesivamente qué brazo accionar para maximizar la recompensa acumulada a lo largo del tiempo, gestionando el compromiso entre exploración y explotación.

El objetivo principal del capítulo es implementar y comparar diferentes algoritmos de resolución sobre el problema del bandido de k -brazos. En primer lugar, se presenta una descripción formal del problema y los conceptos teóricos necesarios para su comprensión. Posteriormente, se describen los algoritmos implementados. Finalmente, se presentan los experimentos realizados y se analizan los resultados obtenidos.

1.2. Desarrollo

El problema del bandido de k -brazos puede definirse formalmente como un proceso de toma de decisiones en el que un agente dispone de un conjunto de k acciones posibles, denominadas brazos. Cada brazo está asociado a una distribución de probabilidad desconocida que genera recompensas al ser seleccionado. En cada instante temporal, el agente debe elegir uno de los brazos y recibe una recompensa extraída de la distribución correspondiente. El objetivo consiste en maximizar la recompensa acumulada durante el proceso de aprendizaje. La principal dificultad radica en encontrar un equilibrio adecuado entre la exploración de acciones poco conocidas y la explotación de aquellas que han mostrado un buen rendimiento.

El estudio del bandido de k -brazos resulta relevante debido a que constituye una simplificación de muchos problemas de aprendizaje por refuerzo más complejos. Aunque no existe una noción explícita de estado ni de transición entre estados, el problema permite introducir conceptos fundamentales relacionados con la estimación de valores, la optimización de políticas y la evaluación de estrategias de aprendizaje.

En este trabajo se han implementado y analizado tres familias principales de algoritmos, ε -Greedy, Upper Confidence Bound (UCB1) y métodos de ascenso de gradiente (softmax). Los algoritmos han sido evaluados sobre diferentes configura-

ciones del problema utilizando distribuciones de recompensa Bernoulli, Binomial y Normal, permitiendo estudiar su comportamiento bajo distintos niveles de incertidumbre y variabilidad. Este último punto representa la principal aportación del trabajo frente a enfoques previos.

1.2.1. Métricas de evaluación

Para analizar el rendimiento de los algoritmos estudiados se han utilizado las siguientes métricas:

- **Recompensa media.** La recompensa media mide el beneficio obtenido por el algoritmo a lo largo del tiempo de aprendizaje. Se define como el promedio de las recompensas observadas en cada instante. r_t representa la recompensa obtenida en el instante t y T el número total de iteraciones

$$r(t) = \frac{1}{T} \sum_{t=1}^T r_t,$$

- **Porcentaje de selección del brazo óptimo:** indica la frecuencia con la que el algoritmo selecciona la acción óptima durante el proceso de aprendizaje. Si $N^*(T)$ representa el número de veces que se selecciona el brazo óptimo hasta el instante T , entonces el porcentaje de selección del brazo óptimo se define como:

$$P_{\text{óptimo}}(T) = \frac{N^*(T)}{T} \times 100,$$

- **Regret acumulado.** El *regret* acumulado cuantifica la pérdida de rendimiento al no seleccionar la acción óptima. Se define como la diferencia entre la recompensa esperada del brazo óptimo (q^*) y la recompensa esperada del brazo seleccionado en cada instante ($\mu(a_t)$):

$$R(T) = \sum_{t=1}^T (q^* - \mu(a_t)),$$

- **Distribución de selección de acciones.** Se analiza mediante histogramas la frecuencia de selección de cada brazo durante el entrenamiento.

1.3. Algoritmos

En este trabajo todos los métodos comparten la misma estructura general de interacción mostrada en el algoritmo 1. Las soluciones se basan en los fundamentos teóricos del libro de Sutton y Barto[1].

La principal diferencia entre los algoritmos estudiados radica en la estrategia utilizada para la **selección del brazo** en cada iteración, es decir, en la forma en que equilibran la exploración de nuevas acciones frente a la explotación de aquellas que han mostrado mejores resultados hasta el momento.

Algorithm 1 Ejecución de experimentos

Require: Bandido B , conjunto de algoritmos A , número de pasos S , número de ejecuciones R

Ensure: Recompensas medias, porcentaje de selecciones óptimas, regret acumulado y estadísticas por brazo

```

1: Inicializar las estructuras para almacenar las métricas
2: for  $r = 1$  to  $R$  do
3:   Inicializar una nueva instancia del bandido  $B$ 
4:   Reiniciar todos los algoritmos  $a \in A$ 
5:   for  $s = 1$  to  $S$  do
6:     for all algoritmo  $a \in A$  do
7:       Seleccionar brazo  $b_s$ 
8:       Obtener recompensa  $r_s$  de  $b_s$ 
9:       Actualizar estimaciones con  $r_s$ 
10:      Registrar recompensa
11:      Actualizar estadísticas de  $b_s$ 
12:      Actualizar métricas
13:    end for
14:  end for
15: end for
16: Calcular medias finales sobre las  $R$  ejecuciones
  
```

Las distintas estrategias utilizadas para la selección del brazo en el problema de los *multi-armed bandits* se describen a continuación.

1.3.1. ϵ -greedy

El método ϵ -greedy introduce aleatoriedad en la selección de acciones para favorecer la exploración. En cada instante t , se genera un valor aleatorio $u \sim \mathcal{U}(0,1)$ y se procede de la siguiente manera:

- ▶ Si $u < \epsilon$, se realiza **exploración**, seleccionando un brazo de forma aleatoria.
- ▶ En caso contrario, se realiza **explotación**, seleccionando el brazo con mayor recompensa media estimada. $Q_t(a)$ representa la estimación de la recompensa media de la acción a en el instante t :

$$a_t = \arg \max_a Q_t(a),$$

El método ϵ -greedy se ha evaluado bajo distintas variantes. En primer lugar, se ha considerado el caso de ϵ constante. Por otro lado, se ha analizado una versión con decaimiento de ϵ , en la que la exploración se reduce progresivamente a medida que avanza el entrenamiento. Finalmente, también se han estudiado variantes que incorporan un barrido inicial de exploración.

1.3.2. Upper Confidence Bound (UCB1)

El método UCB1 selecciona la acción que maximiza una cota superior de confianza, equilibrando la recompensa estimada con un término de exploración dependiente de la incertidumbre.

En cada instante t , la acción seleccionada se define como:

$$a_t = \arg \max_a \left(Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right),$$

donde:

- ▶ $Q_t(a)$ es la recompensa media estimada del brazo a .
- ▶ $N_t(a)$ es el número de veces que se ha seleccionado el brazo a hasta el instante t .
- ▶ t es el número total de acciones realizadas.
- ▶ c es un parámetro que controla el grado de exploración.

1.3.3. Softmax

El método softmax asigna probabilidades de selección a cada acción en función de su valor estimado, favoreciendo aquellas con mayor recompensa esperada, pero manteniendo la exploración de forma probabilística.

La probabilidad de seleccionar una acción a en el instante t se define como:

$$P(A_t = a) = \pi_t(a) = \frac{\exp(Q_t(a)/\tau)}{\sum_{b=1}^k \exp(Q_t(b)/\tau)},$$

donde:

- ▶ $Q_t(a)$ es la recompensa media estimada de la acción a en el instante t .
- ▶ k es el número total de acciones disponibles.
- ▶ τ es el parámetro de temperatura, que controla el nivel de exploración.
- ▶ $\pi_t(a)$ representa la política de selección de acciones.

1.4. Evaluación/Experimentos

Los experimentos se han realizado con el objetivo de estudiar cómo diferentes estrategias de exploración y explotación afectan al rendimiento del agente bajo distintas distribuciones de recompensa (Bernoulli, Binomial y Normal). Para ello, se ha implementado un entorno simulado en Python, considerando diversas configuraciones del problema. Cada experimento se repite múltiples veces para obtener estimaciones robustas de las métricas. Los resultados se evalúan mediante las métricas anteriormente descritas.

Epsilon-greedy

- ▶ Cuando $\epsilon = 0$, el algoritmo tiende a concentrarse demasiado pronto en las acciones inicialmente prometedoras, lo que reduce la capacidad de encontrar el óptimo global.
- ▶ Un valor intermedio como $\epsilon = 0,1$ suele ofrecer un mejor compromiso entre exploración y explotación, aunque no siempre maximiza la recompensa media en todos los escenarios debido a la necesidad de seguir explorando.
- ▶ En la variante ϵ con decaimiento, valores iniciales ligeramente superiores a $\epsilon = 0,1$ tienden a mejorar el rendimiento global del algoritmo en comparación con un ϵ constante, ya que la exploración se reduce progresivamente a medida que avanza el entrenamiento, permitiendo una mayor explotación de las acciones con mejor recompensa estimada.
- ▶ Al incorporar una fase inicial de exploración sobre todos los brazos, el impacto del valor de ϵ se reduce de forma notable, ya que el algoritmo parte de una información inicial más equilibrada entre acciones. Sin embargo, esta menor sensibilidad al parámetro puede disminuir en entornos con distribuciones de recompensa altamente variables, donde resulta necesario mantener la exploración durante un mayor número de iteraciones para evitar una convergencia prematura.

Softmax

- ▶ $\tau = 1$ obtiene el mejor rendimiento global, seguido de $\tau = 2$, lo que indica que un nivel intermedio de exploración es el más adecuado.
- ▶ $\tau = 0,5$ tiende a ser más explotador, lo que acelera la convergencia inicial pero puede penalizar el rendimiento final.
- ▶ En términos comparativos, el método softmax es el que presenta el peor rendimiento global frente a los demás algoritmos analizados, mostrando una menor eficiencia en la convergencia hacia la acción óptima.

UCB1

- ▶ Para los valores analizados (0.25, 0.75 y 1), las diferencias observadas son reducidas.
- ▶ El algoritmo presenta una convergencia estable hacia el brazo óptimo sin necesidad de ajuste fino.
- ▶ En términos globales, UCB1 es el algoritmo más estable y el que obtiene el mejor rendimiento entre todos los métodos analizados, destacando por su equilibrio natural entre exploración y explotación sin necesidad de un ajuste fino de parámetros.

Para todos los experimentos, el rendimiento medio es robusto frente a cambios en la distribución de recompensas (Normal, Binomial o Bernoulli), siempre que el horizonte de simulación sea suficientemente largo. Debido a que no se observan diferencias significativas entre distribuciones en los resultados

finales, se presenta a continuación una tabla (1.1) comparativa con los mejores resultados obtenidos con en el caso de distribución Normal, como representación del comportamiento general de los algoritmos.

Algoritmo	% óptimo	Regret
ϵ -greedy (decaimiento, $\epsilon_0 = 0,25$)	90	155
Softmax ($\tau = 1$)	37	1250
UCB1 ($c = 0,75$)	99	55

Tabla 1.1: Comparación de métricas para la distribución Normal utilizando la mejor configuración de cada algoritmo.

A continuación se presentan las gráficas correspondientes al porcentaje de selección del brazo óptimo (1.1) y al regret acumulado (1.2) para el algoritmo UCB1. En particular, se incluye también el histograma de selección de acciones asociado a la configuración con $c = 0,75$ (1.3). Estas figuras permiten analizar de forma conjunta la capacidad del algoritmo para converger hacia la acción óptima y la distribución de exploración entre los distintos brazos.

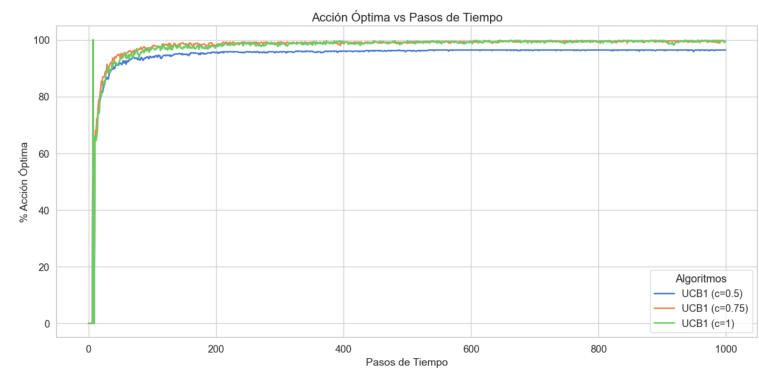


Figura 1.1: Gráfica del porcentaje de acciones óptimas

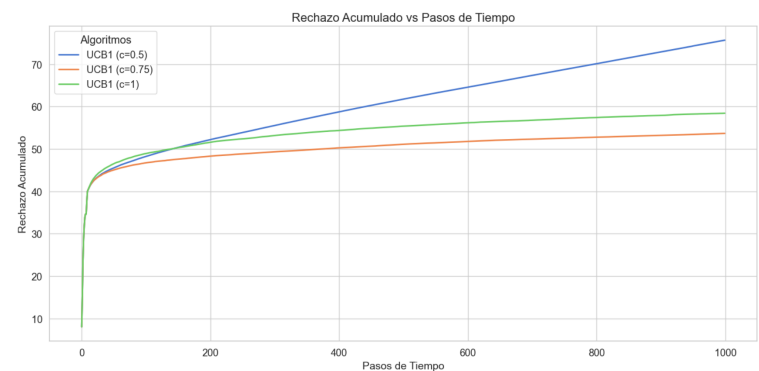


Figura 1.2: Gráfica del regret acumulado

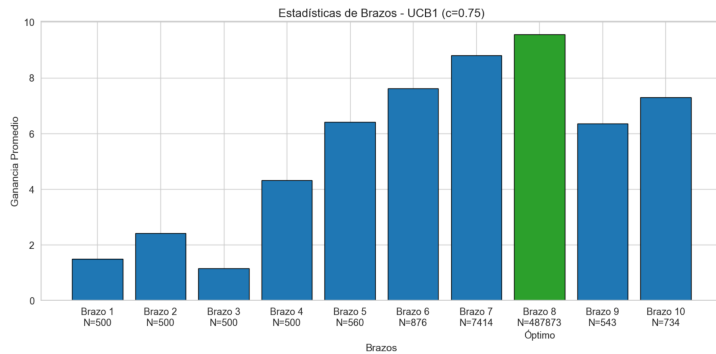


Figura 1.3: Distribución de la elección de brazos con $c = 0,75$

1.5. Conclusiones

En este capítulo se ha estudiado el problema de los *Multi-Armed Bandits*, un caso fundamental del aprendizaje por refuerzo en el que un agente debe comprender de forma práctica el equilibrio exploración-explotación a partir de la interacción con el entorno.

- **Resumen de resultados:** Los algoritmos analizados presentan comportamientos diferenciados en términos de rendimiento y sensibilidad a los parámetros. UCB1 destaca como el método más estable y con mejor rendimiento global, seguido de ϵ -greedy, que muestra buenos resultados pero depende del nivel de exploración seleccionado. Por el contrario, softmax presenta un rendimiento inferior en los escenarios evaluados, siendo más sensible a la elección del parámetro de temperatura.
- **Limitaciones:** El estudio se ha realizado en entornos simulados con un número limitado de configuraciones y sin considerar escenarios no estacionarios.
- **Trabajo futuro:** Como líneas futuras se plantea la extensión a entornos no estacionarios o el uso de métodos bayesianos.

Capítulo 2

Aprendizaje en Entornos Complejos

2.1. Introducción

Los desafíos del aprendizaje por refuerzo son más complejos que el problema del bandido de k-brazos. El estudio de estos entornos resulta fundamental para comprender las limitaciones de los métodos clásicos y la necesidad de técnicas de aproximación de funciones. En este contexto, la biblioteca *Gymnasium* [2] proporciona una colección de entornos de simulación que permiten evaluar algoritmos de aprendizaje por refuerzo.

Este capítulo se organiza en dos partes diferenciadas. En primer lugar, se estudian métodos tabulares aplicados a entornos discretos, donde el espacio de estados permite una representación explícita de la función de valor. En segundo lugar, se analizan métodos de aproximación de funciones aplicados a entornos continuos.

2.2. Desarrollo

Se distinguen dos tipos principales de enfoques según la naturaleza del entorno. Por un lado, los métodos tabulares se aplican a entornos discretos como *Taxi-v4*, donde es posible representar explícitamente la función de valor para cada estado-acción. En este contexto se implementan y analizan algoritmos clásicos como Monte Carlo, SARSA y Q-Learning, los cuales permiten estudiar las diferencias entre métodos *on-policy* y *off-policy*.

Por otro lado, se consideran métodos basados en aproximación de funciones, necesarios para entornos continuos como *LunarLander-v3*. En estos escenarios, la función de valor se aproxima mediante modelos paramétricos, lo que permite generalizar el conocimiento a estados no observados. Entre los métodos estudiados se encuentran SARSA semi-gradient y Deep Q-Learning (DQL), que emplean redes neuronales para aproximar la función de valor-acción.

2.2.1. Entorno Taxi-v4

El entorno *Taxi-v4* forma parte de la colección de entornos de Gymnasium. El objetivo consiste en controlar un taxi dentro de una cuadrícula de 5×5 para recoger un pasajero en una ubicación determinada y transportarlo hasta su destino final.

La representación del estado es discreta y está compuesta por cuatro variables: la posición del taxi, la ubicación actual del pasajero y el destino asignado. La combinación de estas variables genera un total de 500 estados posibles.



Figura 2.1: Entorno Gymnasium taxi-v4

El espacio de acciones está formado por seis acciones discretas:

- Moverse al sur.
- Moverse al norte.
- Moverse al este.
- Moverse al oeste.
- Recoger pasajero (*pickup*).
- Dejar pasajero (*dropoff*).

El agente recibe una recompensa positiva al completar correctamente el transporte del pasajero, mientras que las acciones inválidas o los desplazamientos innecesarios son penalizados.

2.2.2. Entorno LunarLander-v3

El entorno *LunarLander-v3* modela el problema del aterrizaje controlado de un módulo espacial sobre una superficie lunar. A diferencia de *Taxi-v3*, el espacio de estados es continuo, lo que dificulta el uso de métodos tabulares y motiva la utilización de técnicas de aproximación de funciones.

El estado del sistema está representado por un vector de ocho variables continuas:

- Posición horizontal.
- Posición vertical.
- Velocidad horizontal.
- Velocidad vertical.
- Ángulo de inclinación.
- Velocidad angular.
- Contacto de la pata izquierda con el suelo.
- Contacto de la pata derecha con el suelo.

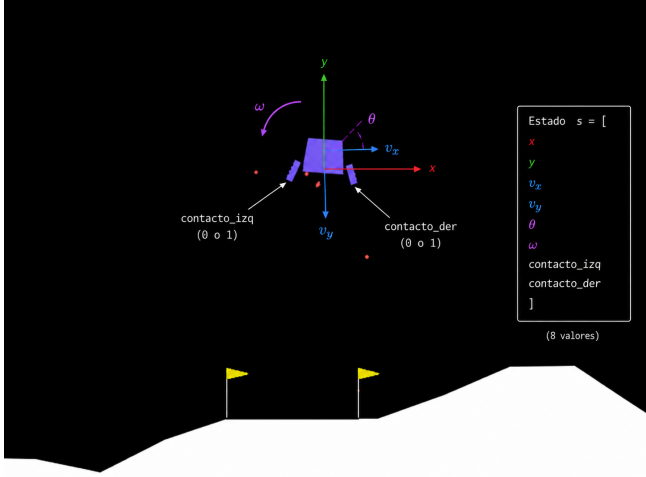


Figura 2.2: Entorno Gymnasium LunarLander-v3

El agente dispone de cuatro acciones posibles:

- ▶ No realizar ninguna acción.
- ▶ Activar el motor principal.
- ▶ Activar el propulsor lateral izquierdo.
- ▶ Activar el propulsor lateral derecho.

La función de recompensa incentiva aterrizajes suaves y estables cerca de la zona objetivo, penalizando colisiones, movimientos bruscos y el consumo excesivo de combustible.

2.2.3. Métricas de evaluación

Para analizar el rendimiento de los algoritmos estudiados se han utilizado las siguientes métricas:

- ▶ **Recompensa por episodio.** La recompensa por episodio mide el retorno obtenido por el agente en cada episodio de interacción con el entorno.
- ▶ **Número de pasos por episodio.** El número de pasos por episodio indica la duración de cada episodio de interacción con el entorno.

2.3. Algoritmos

2.3.1. Monte Carlo

El Algoritmo 2 resume el proceso de entrenamiento seguido por el agente Monte Carlo On-Policy. El agente genera episodios completos siguiendo una política ϵ -greedy y almacena todas las transiciones observadas durante la ejecución.

Una vez alcanzado un estado terminal, se calcula el retorno acumulado asociado a cada visita de los pares estado-acción presentes en la trayectoria generada. A partir de estos retornos, se actualizan las estimaciones de la función de valor-acción $Q(s, a)$ mediante el criterio Every-Visit Monte Carlo, considerando todas las apariciones de cada par estado-acción dentro del episodio.

El algoritmo descrito se ha implementado siguiendo este enfoque tabular para el entorno Taxi-v4, almacenando y actualizando directamente las estimaciones de valor de todos los pares estado-acción visitados durante el entrenamiento.

Algorithm 2 Monte Carlo On-Policy con política ϵ -greedy

Require: Entorno E , agente Monte Carlo M , número de episodios N , factor de descuento γ , parámetros ϵ_0 , ϵ_{min} y $decay$

Ensure: Recompensas por episodio y longitudes de episodio

```

1: for  $e = 1$  to  $N$  do
2:   Reiniciar entorno  $E$ 
3:   Obtener estado inicial  $s_0$ 
4:   while el episodio no haya finalizado do
5:     Seleccionar acción  $a_t$  mediante política  $\epsilon$ -greedy
6:     Ejecutar acción  $a_t$  en el entorno
7:     Observar recompensa  $r_t$  y siguiente estado  $s_{t+1}$ 
8:     Almacenar transición  $(s_t, a_t, r_t, s_{t+1})$ 
9:      $s_t \leftarrow s_{t+1}$ 
10:  end while
11:  for all pares estado-acción  $(s, a)$  visitados do
12:    Actualizar  $Q(s, a)$  usando Every-Visit Monte Carlo
13:  end for
14:  Aplicar decaimiento de  $\epsilon$ 
15:  Registrar métricas
16: end for

```

2.3.2. SARSA/SARSA Semi-Gradient

El Algoritmo 3 describe el proceso de entrenamiento seguido por el agente SARSA. Este método pertenece a la familia de algoritmos de diferencias temporales y se caracteriza por actualizar las estimaciones de la función de valor-acción durante la propia interacción con el entorno, sin necesidad de esperar a la finalización de un episodio completo.

La actualización de la función $Q(s, a)$ se realiza utilizando tanto la recompensa inmediata como el valor estimado del siguiente par estado-acción efectivamente elegido por el agente. En su versión tabular (la utilizada para Taxi-v4), la actualización viene dada por la expresión:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

donde α representa la tasa de aprendizaje y γ el factor de descuento.

Debido a que la acción utilizada en la actualización es la misma que genera la experiencia futura, SARSA se clasifica como un método *on-policy*.

En el caso de Semi-Gradient SARSA (entorno continuo LunarLander-V3), el esquema general de aprendizaje se mantiene sin cambios. La principal diferencia radica en la representación de la función de valor-acción, ya que en lugar de almacenar explícitamente los valores $Q(s, a)$ en una tabla, estos se aproximan mediante una red neuronal parametrizada. La actualización viene dado por la siguiente expresión donde $\mathbf{w}$ representa el conjunto de parámetros de la red neuronal:

$$\delta_t = r_t + \gamma Q(s_{t+1}, a_{t+1}; \mathbf{w}) - Q(s_t, a_t; \mathbf{w})$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha, \delta_t, \nabla_{\mathbf{w}} Q(s_t, a_t; \mathbf{w})$$

Algorithm 3 SARSA con política ϵ -greedy

Require: Entorno E , agente SARSA M , número de episodios N , factor de descuento γ , tasa de aprendizaje α , parámetros ϵ_0 , ϵ_{min} y $decay$

Ensure: Recompensas por episodio y longitudes de episodio

```

1: for  $e = 1$  to  $N$  do
2:   Reiniciar entorno  $E$ 
3:   Obtener estado inicial  $s_0$ 
4:   Seleccionar acción inicial  $a_0$  mediante política  $\epsilon$ -greedy
5:   while el episodio no haya finalizado do
6:     Ejecutar acción  $a_t$  en el entorno
7:     Observar recompensa  $r_t$  y siguiente estado  $s_{t+1}$ 
8:     Seleccionar acción  $a_{t+1}$  mediante política  $\epsilon$ -greedy
9:     Actualizar  $Q(s_t, a_t)$ 
10:    Actualizar métricas del episodio
11:     $(s_t, a_t) \leftarrow (s_{t+1}, a_{t+1})$ 
12:  end while
13:  Aplicar decaimiento de  $\epsilon$ 
14:  Registrar métricas
15: end for

```

2.3.3. Q-Learning/Deep Q-Learning

El Algoritmo 4 describe el proceso de entrenamiento seguido por el agente Q-learning (Taxi-V4). En cada estado, el agente selecciona una acción siguiendo una política ϵ -greedy y la ejecuta en el entorno. Tras observar la recompensa obtenida y el siguiente estado, se actualiza inmediatamente la estimación de la función $Q(s, a)$ utilizando la recompensa recibida y la mejor acción estimada para el siguiente estado. La actualización se realiza mediante la siguiente expresión:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$$

donde α representa la tasa de aprendizaje y γ el factor de descuento.

A diferencia de SARSA, que emplea la acción realmente seleccionada en el siguiente estado para realizar la actualización, Q-learning utiliza el valor máximo estimado entre todas las acciones posibles. Por este motivo, se clasifica como un método *off-policy*.

En el caso de Deep Q-Learning/DQL (LunarLander-V3), el esquema general de entrenamiento se mantiene esencialmente igual. La principal diferencia radica en que la función de valor-acción ya no se representa mediante una tabla, sino mediante una red neuronal que aproxima los valores $Q(s, a; \mathbf{w})$.

Además, las transiciones observadas durante la interacción con el entorno no se utilizan inmediatamente para actualizar la red. En su lugar, se almacenan en una memoria de repetición, desde la que se extraen de forma aleatoria mini-lotes de experiencias para realizar el entrenamiento.

Algorithm 4 Q-learning con política ϵ -greedy

Require: Entorno E , agente Q-learning M , número de episodios N , factor de descuento γ , tasa de aprendizaje α , parámetros ϵ_0 , ϵ_{min} y $decay$

Ensure: Recompensas por episodio y longitudes de episodio

```

1: for  $e = 1$  to  $N$  do
2:   Reiniciar entorno  $E$ 
3:   Obtener estado inicial  $s_0$ 
4:   while el episodio no haya finalizado do
5:     Seleccionar acción  $a_t$  mediante política  $\epsilon$ -greedy
6:     Ejecutar acción  $a_t$  en el entorno
7:     Observar recompensa  $r_t$  y siguiente estado  $s_{t+1}$ 
8:     Actualizar  $Q(s_t, a_t)$ 
9:     Actualizar métricas del episodio
10:     $s_t \leftarrow s_{t+1}$ 
11:  end while
12:  Aplicar decaimiento de  $\epsilon$ 
13:  Registrar métricas
14: end for

```

2.4. Evaluación/Experimentos**2.4.1. Tabular: Taxi-V4**

Los experimentos se han realizado sobre los algoritmos Monte Carlo, SARSA y Q-Learning. Para ello, cada agente se ha entrenado durante el mismo número de episodios evaluando la evolución de la recompensa media obtenida durante el entrenamiento. En todos los casos se emplea una política ϵ -greedy con decaimiento. Sin embargo, en Monte Carlo se utiliza un decaimiento más lento para mantener una exploración elevada durante un mayor número de episodios, ya que este algoritmo necesita episodios completos para actualizar sus estimaciones.

Monte Carlo

- ▶ Durante una parte importante del entrenamiento se obtienen recompensas muy negativas debido a la elevada exploración inicial y a la necesidad de completar episodios antes de actualizar la política. Como consecuencia, el agente realiza numerosas trayectorias largas e ineficientes.
- ▶ La mejora de la recompensa media es significativamente más lenta que en los algoritmos de diferencia temporal, requiriendo un mayor número de episodios para comenzar a aprender una política útil.
- ▶ Se observan fluctuaciones incluso en fases avanzadas del entrenamiento debido a la exploración residual y a que las actualizaciones se realizan únicamente al finalizar cada episodio, lo que incrementa la varianza del aprendizaje.

SARSA

- ▶ Al igual que Monte Carlo, comienza con recompensas muy negativas como consecuencia de la elevada exploración inicial. Sin embargo, la recompensa media mejora rápidamente durante los primeros episodios, mostrando una convergencia más rápida y estable que Monte Carlo gracias a las actualizaciones por diferencia temporal.

- ▶ El algoritmo alcanza valores próximos a la recompensa óptima, manteniendo un comportamiento estable durante el resto del entrenamiento.

Q-Learning

- ▶ Desde los primeros episodios obtiene recompensas relativamente elevadas en comparación con los demás algoritmos, mostrando una rápida capacidad de aprendizaje.
- ▶ El algoritmo presenta un comportamiento estable durante la mayor parte del entrenamiento, con pequeñas fluctuaciones debidas únicamente a la exploración residual.
- ▶ En términos generales, Q-Learning es el algoritmo que presenta la convergencia más rápida y el mejor rendimiento entre los métodos analizados.

En los tres algoritmos se observa una mejora progresiva del rendimiento a medida que avanza el entrenamiento. SARSA y Q-Learning alcanzan finalmente una recompensa media cercana a 6, correspondiente al comportamiento óptimo en el entorno Taxi-v4 para la semilla fijada, y mantienen dicho rendimiento de forma estable durante las últimas etapas del aprendizaje. En el caso de Monte Carlo, aunque también muestra una tendencia ascendente y puede aproximarse al valor óptimo, algunas ejecuciones no llegan a converger completamente a una recompensa 6.

2.4.2. Continuo: LunarLander-V3

Los experimentos se han realizado sobre los algoritmos Deep Q-Learning y SARSA semi-gradient en el entorno LunarLander-v3. Ambos agentes se han entrenado durante el mismo número de episodios utilizando una política ϵ -greedy con decaimiento, evaluando la evolución de la recompensa obtenida durante el entrenamiento. Los resultados permiten comparar la velocidad de aprendizaje, la estabilidad del entrenamiento y la calidad de la política aprendida en un entorno continuo de mayor complejidad que Taxi-v4.

Deep Q-Learning

- ▶ Durante los primeros episodios se obtienen recompensas claramente negativas debido a que el agente comienza el entrenamiento sin conocimiento previo del entorno y con un elevado nivel de exploración ($\epsilon = 1$), produciendo aterrizajes inestables y numerosas penalizaciones.
- ▶ A medida que avanza el entrenamiento, la recompensa media mejora progresivamente, aunque se observan fases de inestabilidad e incluso ligeras caídas intermedias antes de alcanzar un comportamiento consistente.
- ▶ Finalmente, el algoritmo converge hacia recompensas superiores a 200, correspondientes a una política capaz de realizar aterrizajes satisfactorios de forma habitual.
- ▶ El entrenamiento presenta una elevada variabilidad durante prácticamente todo el proceso, con fluctuaciones atribuibles a la exploración residual.

SARSA semi-gradient

- ▶ Al comienzo del entrenamiento el agente obtiene recompensas negativas, consecuencia de la exploración inicial y del desconocimiento del entorno.
- ▶ La recompensa media aumenta de forma progresiva y considerablemente más estable que en Deep Q-Learning, mostrando una evolución prácticamente continua durante el entrenamiento.
- ▶ El algoritmo converge hacia recompensas cercanas a 200, aprendiendo una política eficaz para completar el aterrizaje.
- ▶ Aunque persisten pequeñas fluctuaciones provocadas por la exploración residual, el comportamiento global resulta más suave y estable que el observado en Deep Q-Learning.

Ambos algoritmos consiguen aprender una política capaz de resolver el entorno LunarLander-v3, alcanzando finalmente recompensas próximas a 200 (cuando el episodio se considera superado). Sin embargo, se aprecian diferencias claras en la estabilidad del entrenamiento. Deep Q-Learning presenta una mayor variabilidad y oscilaciones durante el proceso de aprendizaje, mientras que SARSA semi-gradient muestra una evolución más suave y estable, alcanzando la convergencia de forma más progresiva.

A continuación se presentan las gráficas de evolución de la recompensa para ambos algoritmos.

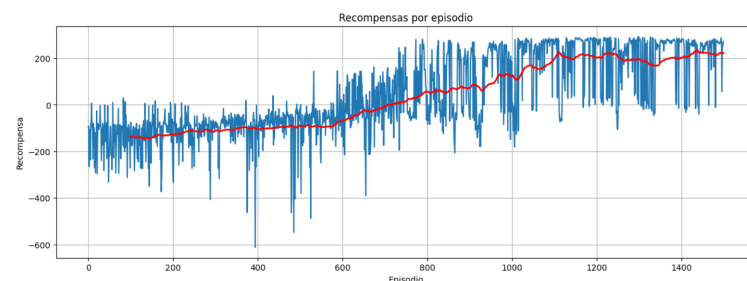


Figura 2.3: Recompensa por episodio Sarsa semi gradient

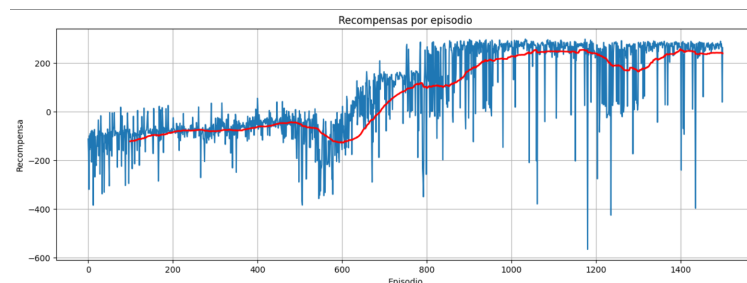


Figura 2.4: Recompensa por episodio Deep Q-Learning

2.5. Conclusiones

En este capítulo se han estudiado distintos algoritmos de aprendizaje por refuerzo aplicados a dos entornos con características diferentes. Taxi-v4, basado en un espacio de estados discreto, y LunarLander-v3, que requiere aproximación de funciones mediante redes neuronales o métodos semi-gradiente.

- ▶ **Resumen de resultados:** En Taxi-v4, los algoritmos basados en diferencia temporal obtienen el mejor comportamiento, destacando Q-Learning por su rápida convergencia y estabilidad, seguido de SARSA. En LunarLander-v3, tanto Deep Q-Learning como SARSA semi-gradient alcanzan políticas capaces de resolver el entorno. No obstante, SARSA semi-gradient muestra un aprendizaje más estable, mientras que Deep Q-Learning presenta una mayor variabilidad.
- ▶ **Limitaciones:** El estudio se ha realizado sobre dos entornos concretos empleando una única configuración de hiperparámetros y una política ϵ -greedy con decaimiento. Además, los resultados se han obtenido utilizando una única semilla de ejecución, por lo que no se ha evaluado la variabilidad asociada a diferentes inicializaciones aleatorias ni la robustez de los algoritmos frente a ellas. Tampoco se ha analizado la influencia de distintas arquitecturas de red.
- ▶ **Trabajo futuro:** Como líneas futuras se plantea evaluar algoritmos más avanzados, como Double DQL, así como estudiar su comportamiento en entornos de mayor complejidad y con espacios de estados y acciones más exigentes.

Bibliografía

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 2 ed., 2018.
- [2] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024. <https://gymnasium.farama.org>.